

SPECTROview: An open-source application for spectroscopic data processing, fitting, and visualization

Van Hoan Le ^{a,*}

^a Univ. Grenoble Alpes, CEA, LETI, Grenoble 38000, France

* Corresponding author. E-mail address: van-hoan.le@cea.fr

Abstract

SPECTROview is a free, open-source desktop application for interactive processing, curve fitting, and visualization of spectroscopic data. Written in Python with a PySide6 graphical interface, it addresses the needs of researchers and engineers who work with discrete spectra and hyperspectral datasets, including two-dimensional maps and semiconductor wafer maps, produced by spectroscopic techniques (such as Raman spectroscopy, photoluminescence, etc.). The application is organized into three core workspaces (Spectra, Maps, and Graphs) that cover the complete analytical workflow: data import from multiple instrument formats, preprocessing with interactive baseline correction, multi-peak curve fitting, and publication-quality statistical plotting. A central contribution is the Vectorized Batch Fit Engine (VBF), a batched Levenberg–Marquardt optimizer that fits all spectra of a dataset simultaneously using vectorized NumPy operations and analytical Jacobians, achieving up to 60× speed up over conventional lmfit based per-spectrum approaches. SPECTROview is cross-platform, installable via pip, and released under the GPL-3.0 license.

Keywords

Spectroscopy; VBF engine; Hyperspectral data; Peak fitting; Data visualization; Open-source software

Metadata

Nr	Code metadata description	Metadata
C1	Current code version	26.24.1
C2	Permanent link to code/repository used for this code version	https://github.com/CEA-MetroCarac/SPECTROview
C3	Legal code license	GNU General Public License (GPL-3.0)
C4	Code versioning system used	git
C5	Software code languages, tools and services used	Python
C6	Compilation requirements, operating environments and dependencies	Python between (3.8 – 3.12); PySide6, NumPy (< 2.0), SciPy, Pandas, Matplotlib, Seaborn (cf. requirements.txt and pyproject.toml for the full list).
C7	If available, link to developer documentation/manual	https://CEA-MetroCarac.github.io/SPECTROview/
C8	Support email for questions	van-hoan.le@cea.fr

1. Motivation and significance

Spectroscopic techniques such as Raman scattering and photoluminescence (PL) are routine characterization methods widely used across diverse fields including materials science, chemistry, biology, geology, and semiconductor manufacturing. Modern spectrometers routinely acquire hyperspectral datasets, recording a full spectrum at every point on a two-dimensional spatial grid, yielding maps with thousands to tens of thousands of spectra per sample. Extracting quantitative information from these datasets requires a multi-step workflow encompassing file import, spectral preprocessing (e.g., range selection, baseline correction), peak-shape fitting, result aggregation, and statistical visualization.

In practice, this workflow is often fragmented across multiple software solutions. Data acquisition relies on vendor-specific proprietary software (e.g., Renishaw WiRE, Horiba LabSpec, etc.), which typically offers limited fitting flexibility, requires tedious manual data collecting, lacks batch and custom statistical plotting, and imposes restrictive licensing that limits broader scientific access. General-purpose open-source fitting tools, such as fityk^[1], provide flexible curve fitting with parameter constraints but lack native support for hyperspectral map visualization and integrated plotting. More recent Python-based tools, such as fitspy^[2], wrap the lmfit library^[3] to support 2D map processing; however, their reliance on sequential, Python-level iteration for the fitting loop becomes prohibitively slow for large datasets. While parallelism via multithreading or multiprocessing can partially mitigate this bottleneck, Python's Global Interpreter Lock (GIL) prevents true concurrent execution of CPU-bound threads, and multiprocessing introduces significant overhead from inter-process communication and memory serialization, limiting its scalability on standard workstations with moderate core counts. Library-oriented solutions like RamanSPy^[4] offer powerful preprocessing and machine-learning pipelines but lack a graphical user interface (GUI) for interactive peak fitting. PyFasma provides a Jupyter Notebook-based API for Raman preprocessing and multivariate analysis (PCA, PLS-DA), with spectral deconvolution powered by LMFIT. However, it lacks a graphical user interface, native hyperspectral map

support, and batched fitting capabilities ^[5]. Finally, environments like Orange/Quasar ^[6] provide visual programming workflows tailored for chemometric classification rather than explicit curve-fitting analysis.

Currently, no single free and open-source application combines (i) multi-format data import, (ii) interactive baseline and peak fitting with parameter constraints, (iii) native support both for 2D map and wafer map data processing and visualization, (iv) high-performance batched fitting, and (v) integrated publication-quality statistical plotting.

SPECTROview was developed at the Platform for Nanocharacterisation (PFNC) of CEA-LETI to address this gap. Designed by researcher-engineers for researcher-engineers, its feature set reflects the demands of processing the massive volumes of spectroscopic data generated daily in both research laboratories and semiconductor cleanroom production scale. The software architecture prioritizes both usability and computational performance. The central technical contribution is the Vectorized Batch Fit Engine (VBF), a novel batched Levenberg–Marquardt optimizer tailored specifically for large-scale hyperspectral data. While the mathematical concept of batched optimization is utilized in deep learning frameworks for neural network training, existing open-source spectroscopy tools remain constrained by sequential, one-by-one spectrum optimization. In contrast, the VBF engine formulates the fitting of N spectra as a single tensor-level linear-algebra problem using pure NumPy. Despite this batched formulation, each spectrum converges independently. The optimizer maintains per-spectrum damping factors and a boolean convergence mask that progressively excludes converged spectra, ensuring near-zero overhead for early convergers. This approach achieves up to 60× speedups compared to per-spectrum methods while preserving independent convergence behavior for each individual spectrum.

2. Software description

2.1. Software architecture

SPECTROview is implemented in Python and uses PySide6 (Qt 6)^[7] for its graphical user interface. The application follows a strict Model-View-ViewModel (MVVM) architecture that separates data management (Model), business logic and application orchestration (ViewModel), and user interface layout (View). This modular design enables independent testing of the numerical core and simplifies future extensions of the user interface. Figure 1 presents a simplified overview of the SPECTROview architecture, illustrating the different modules and their interactions. A more detailed description of the architecture is available in the developer guide: <https://cea-metrocarac.github.io/SPECTROview/developer/>

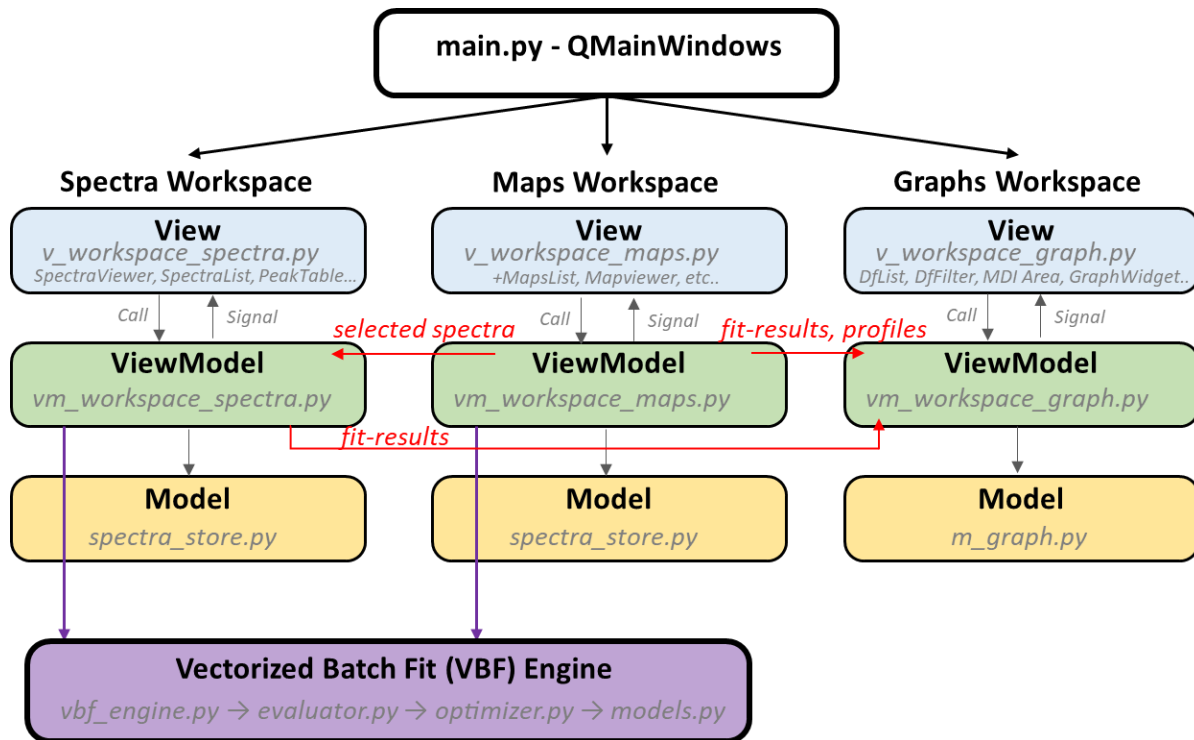


Figure 1: Simplified architecture diagram of SPECTROview with the three workspaces (Spectra, Maps, Graphs), their MVVM layers, and inter-workspace communication.

Figure 2 present the main application's user-interface organized into three workspaces, each implemented as a tab in the main window:

- Spectra Workspace: manages discrete spectra including loading, preprocessing, interactive peak fitting, and fit-result collection.
- Maps Workspace: inherits and extends the Spectra Workspace to handle hyperspectral datasets, adding spatial heatmap visualization (2Dmap and wafer maps), with dedicated listboxes to navigate between multiples loaded hyperspectral datasets and individual spectra within each dataset.
- Graphs Workspace: provides a dataframe statistical plotting environment with support for eight plot styles, dynamic filtering. User can thus work on a single datasheet for multiples plots.

Cross-workspace communication is achieved through Qt signal–slot connections and dependency injection wired at application startup, enabling a seamless workflow across the platform. For example:

- Best-fit results generated in the Spectra or Maps Workspace can be transferred directly to the Graphs Workspace for immediate statistical visualization, detailed comparison, and further analysis.
- Line profiles (extracted by selecting two points on a heatmap of selected 2D map) can be sent directly to the Graphs Workspace, including both the raw data and the generated plot.

- Individual spectra from a 2D map can be transferred to the Spectra Workspace for closer inspection and inter-map comparison, without requiring file-based export.

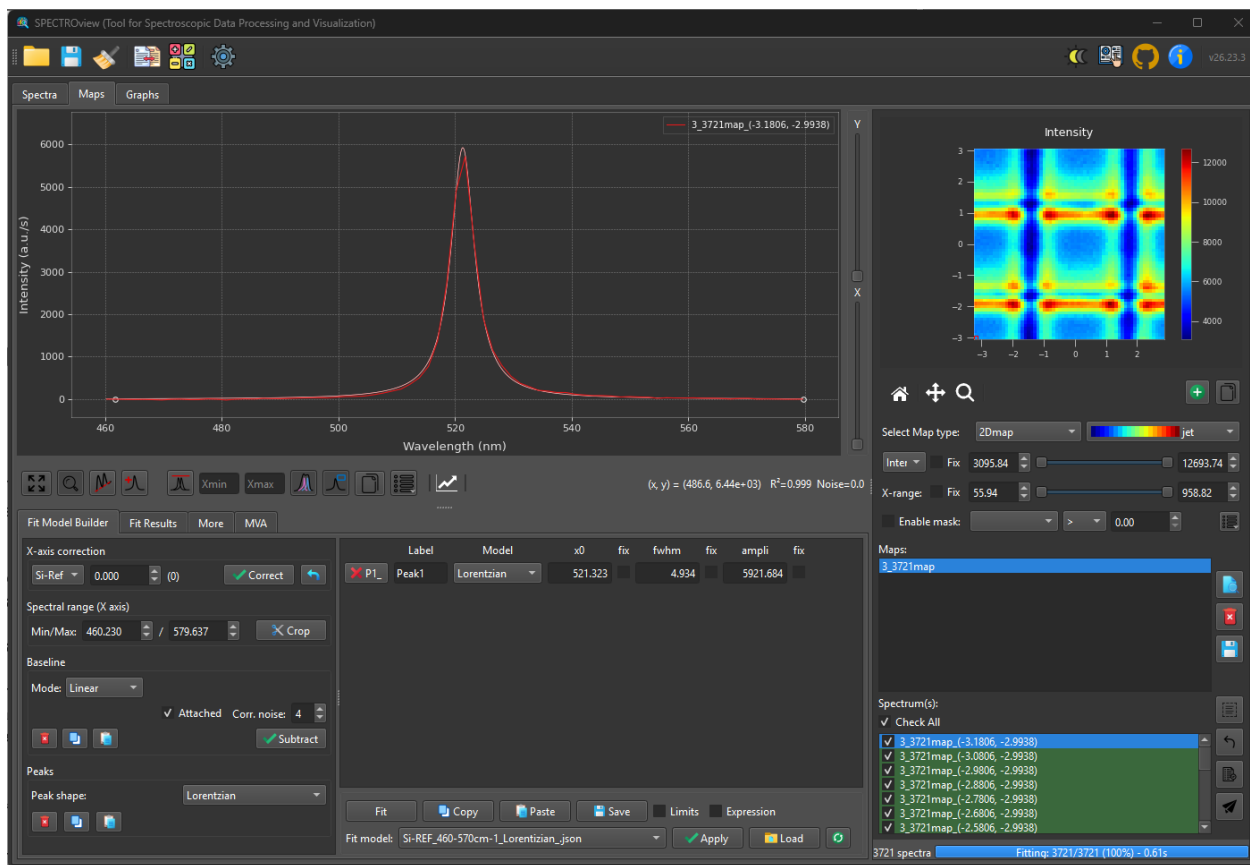


Figure 2: SPECTROview's main user interface organized into three workspaces: Spectra, Maps, Graph (animated figure can be found in project's [documentation page](#)).

2.2. Software functionalities

2.2.1 Data import

Table 1 lists all file formats supported by SPECTROview. Examples of all supported data formats are available in the GitHub repository folder : [examples/spectroscopic_data](#)

Table 1: Supported file formats

Format	Extension	Loader	Data type
Delimited text	.txt, .csv	Auto-detected delimiter	Single spectra and maps
Renishaw WIRE	.wdf	renishawWiRE library	Single spectra and maps
SPC	.spc	Built-in binary reader	Single spectra and maps
TRPL	.dat	Built-in parser	Time-resolved decays
Tabular data	.xlsx, .csv	Pandas & Openpyxl	DataFrames for plotting

2.2.2 Preprocessing

The preprocessing pipeline operates on working copies of the raw data, preserving the original arrays for non-destructive analysis. Available operations include:

- X-axis correction (optional): linear shift of the x axis.
- Spectral range selection: interactive cropping of the x-axis range.
- Baseline correction: both manual (linear, polynomial with user-placed anchor points) and automatic (arPLS, airPLS, asLS, modpoly) baseline subtraction. Automatic baseline algorithms are provided by the pybaselines package ^[8] which leverages optimized NumPy/SciPy solvers ^[9], and can be previewed in real time before subtraction..

All prementioned preprocessing operations are vectorized across the loaded dataset and support both single-spectrum (maps) and all spectra (all maps) application modes.

2.2.3 Multiple peaks fitting with VBF Engine

Users define one or multiple peak model(s) interactively by clicking on the spectrum to place peaks. Each peak is assigned a line shape, and listed on a peak table. Initial parameter guesses are estimated automatically, and parameter bounds and inter-parameter expressions can be set through this peak table interface.

The fitting is performed by the VBF engine, a custom batched Levenberg–Marquardt optimizer ^[10] implemented in the *fit_engine/* folder of the project. Rather than iterating over spectra in a Python loop, the VBF engine formulates the simultaneous fitting of N spectra as a set of coupled tensor operations:

- The N observed spectra are stacked into an $(N \times M)$ data matrix, where M is the number of spectral points.
- Initial parameter guesses are assembled into an $(N \times K)$ matrix, where K is the number of free parameters, with per-spectrum amplitude scaling.
- Model evaluation and Jacobian computation produce $(N \times M)$ and $(N \times M \times K)$ tensors, respectively.
- Matrix multiplications and transpositions for the normal equations ($J^T J$ and $J^T r$) are assembled and performed via *np.einsum* and solved in a single call to *np.linalg.solve* (dispatching to LAPACK) ^[11], yielding the parameter updates for all spectra simultaneously.

All built-in peak models have analytical Jacobians, eliminating the 2K additional function evaluations per iteration that finite-difference Jacobian estimation would require. Despite the batched linear algebra, each spectrum converges independently, i.e., the optimizer maintains a per-spectrum damping factor and a boolean convergence mask that progressively excludes converged spectra from subsequent iterations.

Additional performance optimizations include: (i) a template-based model application that pre-computes Model objects once and clones only lightweight parameter dictionaries; (ii) batch preprocessing that computes range masks and baseline indices once for spectra sharing the

same x-axis; (iii) an adaptive solver that selects between batched `np.linalg.solve`^[11] and Cholesky decomposition (`cho_solve`)^[11] depending on the matrix dimensions; and (iv) noise-aware weight masking that zeros out data points below an estimated noise floor, reducing effective parameter count and preventing ghost-peak artifacts.

Fit models (comprising peak definitions, baseline settings, and parameter constraints) can be copied, and pasted between spectra or maps, or can be saved and reused later, enabling reproducible analysis templates.

2.2.4 Hyperspectral map visualization

The Maps Workspace loads hyperspectral datasets and renders them as interactive heatmaps (Figure 2). The Z-axis (color dimension) can display raw integrated intensity, integrated area, or any fitted parameter once they are collected (cf. section 2.2.5).

Users can select individual (or several) spectra by (hold Ctrl +) clicking on the heatmap, select regions by rectangle-drag, or extract spatial line profiles between two selected points. Selected spectra is plotted and shown in spectra plot widget for inspection and fitting; line profiles are can be extracted and forwarded to the Graphs Workspace as DataFrames.

For semiconductor applications, SPECTROview supports wafer map rendering using `scipy.griddata` interpolation onto a regular grid with a circular mask.

2.2.5 Best-fit data collection and export

Once curve fitting has been applied across a set of discret spectra (Spectra Workspace) or entire hyperspectral maps (Maps Workspace), SPECTROview provides a streamlined interface for aggregating, transforming, and exporting the results. Clicking the “Collect” button in the “Fit Results” tab instantly compiles all best-fit parameters (peak positions, amplitudes, fwhm, ...) into a single, sortable Pandas DataFrame, displayed in the GUI for direct visualization (Figure 3).

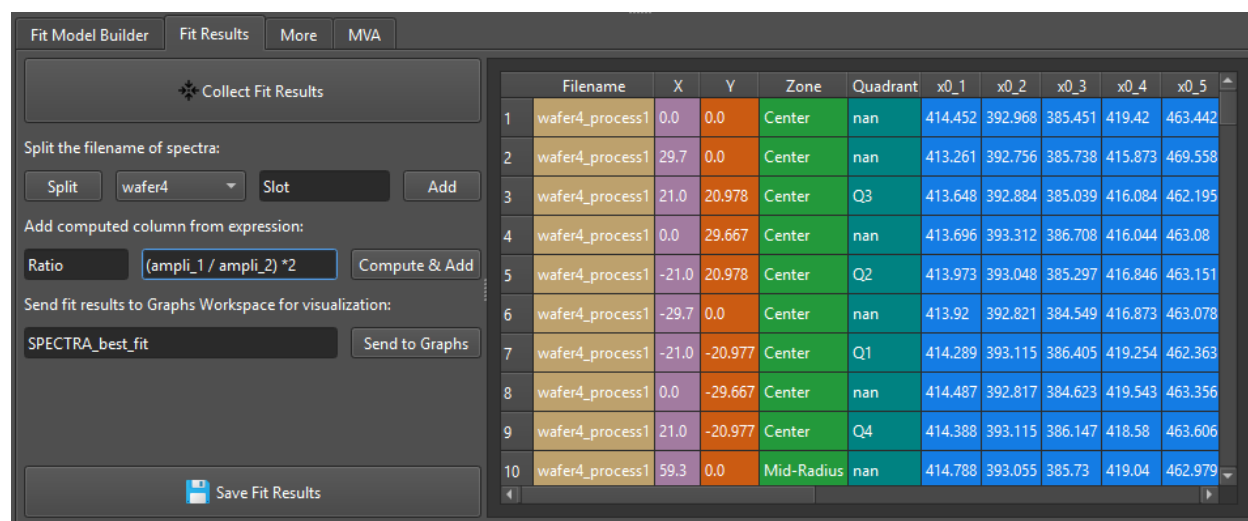


Figure 3: The interface of FitResult tab

A built-in filename-splitting tool can automatically extract metadata embedded in file names (e.g., sample identifiers, process conditions, or measurement temperatures) and map them into

additional DataFrame columns, eliminating the need for manual annotation. This feature is highly advantageous for users who employ strategic file naming conventions during data acquisition.

Users can also define new derived columns using mathematical expressions on existing fitted parameters (e.g., computing a peak shift between two peaks, or an intensity ratio of two or more peaks), with support for standard arithmetic operators (+, -, *, /, **). An example is showed in Figure 3.

The resulting DataFrame can be (i) saved within the workspace for persistent access, (ii) sent directly to the Graphs Workspace for immediate statistical visualization (e.g., box plots, trendlines, wafer maps, etc.), or (iii) exported as an Excel (`.xlsx`) or CSV (`.csv`) file for use in external tools. This single-click collection and in-application transformation workflow removes the intermediate file-export steps that are typically required by other spectroscopy tools, significantly accelerating the path from raw data to publication-ready figures.

2.2.5 Statistical plotting

The Graphs Workspace operates on Pandas DataFrames loaded either from files or transferred from other workspaces. It supports eight plot types: *point*, *scatter*, *box*, *bar*, *line*, *trendline*, *wafer*, and 2D map plots. Visualizations are rendered using *Seaborn* ^[12] and *Matplotlib* ^[13] within a Multiple Document Interface (MDI, Qt), enabling the simultaneous display and comparison of multiple plots.

Each plot can use up to three Y-axes, a dynamic filter system based on *pd.DataFrame.query()*, and per-series legend customization (*label*, *color*, *marker*). A graph customization dialog provides axis limits, annotations, and export to clipboard or image file.

For wafer datasets containing a *Slot* column, a batch wafer-plotting mode automatically generates multiple selected wafer plots at once. In addition, the intelligent “collect fit results” feature can automatically identify data originating from different wafer regions, such as quadrants or radial zones (center, mid-radius, and edge), based on the wafer diameter and XY coordinates of spectra (an example is showed in results data table in Figure 3). This functionality simplifies statistical analysis and comparison across multiple wafers and wafer regions using a single dataset, without requiring additional data cleaning or post-processing.

2.2.6 Multivariate analysis

SPECTROview includes built-in multivariate analysis capabilities: Principal Component Analysis (PCA) via singular value decomposition and Non-negative Matrix Factorization (NMF) via Lee-Seung multiplicative update rules ^[14]. Both methods operate on preprocessed spectra and produce scree plots, loadings, and score visualizations. Results can be exported to the Graphs Workspace for further analysis.

2.2.7 Save and resumes works

SPECTROview uses dedicated native file formats to persist the complete analysis state of each workspace. Clicking the "Save" button in the toolbar generates a workspace-specific archive:

- *.spectra* : stores all loaded 1D spectra, baseline configurations, fit models, and aggregated fit results.

- *.maps* : stores the raw 2D/wafer map data, instrument metadata, associated spectra, and point-by-point fit results.

- *.graphs* : stores all loaded DataFrames, plot configurations, and annotations.

Spectra and Maps archives use the ZIP format containing a JSON metadata file and compressed NumPy arrays, ensuring fast serialization of large datasets. The Graphs archive uses a gzip-compressed JSON format.

To resume a previous session, the user clicks "Open" and selects a saved workspace file (*.spectra*, *.maps*, *.graphs*), or simply drags it onto the main window. SPECTROview automatically detects the format, switches to the corresponding workspace, and reconstructs the dataset and analysis state exactly as it was left. This mechanism ensures full reproducibility and facilitates collaboration: a colleague can open a shared *.maps* file and inspect every fitted spectrum, parameter constraint, and heatmap configuration without repeating any processing step

2.2.8 Other Utilities

In addition to its three core workspaces, SPECTROview integrates some auxiliary modules accessible from the main menu bar:

- [Quick Calculators](#) (Appendix A): A set of interactive scientific calculators designed for routine spectroscopy use in the laboratory. These include calculations for laser spot size and depth of focus, as well as optical penetration depth estimates based on laser wavelength and extinction coefficient, with direct access to the refractiveindex.info database and accompanying pedagogical illustrations. This module also includes a Unit Converter, enabling bidirectional conversions between wavelength (nm), energy (eV), and wavenumber (cm^{-1}), as well as Raman shift calculations for a given laser excitation wavelength.
- Hyperspectral Data Converter : A tool for converting hyperspectral text files into formats supported by SPECTROview.

These utilities are implemented as lightweight, self-contained dialog modules that can be invoked at any point during an analysis session without interrupting the current workspace state.

3. Illustrative examples

This section demonstrates a representative workflow using example datasets distributed with SPECTROview in the */examples/* folder of the GitHub repository.

An intuitive User Manual with animated demo images of different functionality is integrated in the application (accessible via UserManual button in the MenuBar of the main GUI), or an online version here: https://cea-metrocarac.github.io/SPECTROview/user_manual/

3.1. Fitting discreet spectra in Spectra workspace

Once spectra are loaded, user can process the spectra through a sequence of steps: (i) defining a spectral range of interest, (ii) performing baseline correction using either manual or automatic mode, (iii) defining one or more peaks, and (iv) performing the fit.

Peaks are placed interactively by clicking on the spectrum plot. Each peak can be further configured by adjusting its line shape, parameter bounds, or inter-parameter expressions.

Once all peaks are defined, clicking "Fit" dispatches the spectra to the VBF Engine, which fits the selected spectra simultaneously. The fitted curves, individual peak contributions, and residuals are displayed in the spectrum viewer. Users can also load a predefined fit model from a previous session and click the "Apply" button to execute the entire fitting pipeline in a single action.

Figure 4 presents an example of a fitted MoS_2 spectrum with multiple peaks that are directly copied from the application. Users can adjust several parameters before copying such as: figure aspect ratio, show peak labels, add grid lines, modify legend entries or colors of each spectrum, and configure several other display options. See the User Manual for a complete list of available view options.

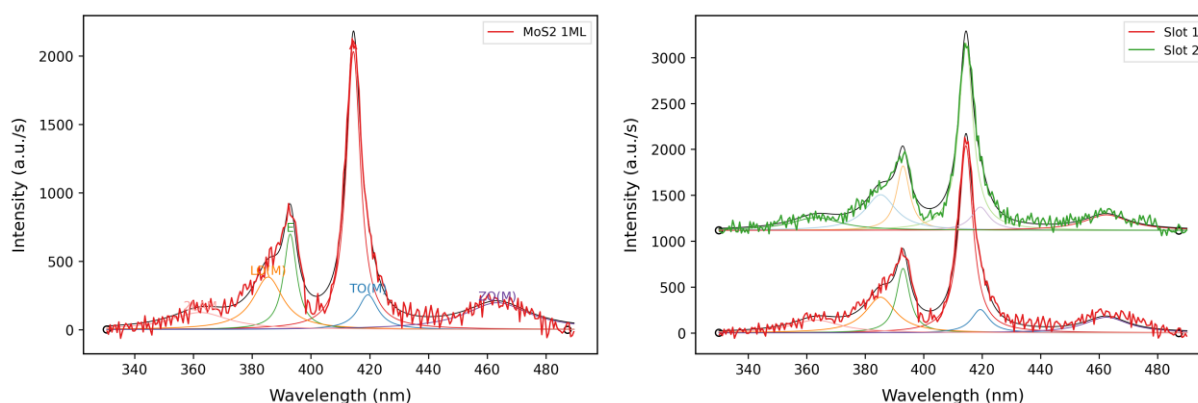


Figure 4: (a) Example of a MoS_2 spectrum after baseline subtraction and multi-peaks fitting. (b) Multiple spectra selection and plot for precise inspection and comparison.

3.2. Analyzing a hyperspectral map in Maps Workspace

As described in Section 2.1, the Maps Workspace inherits all core features of the Spectra Workspace, with adaptations to the user interface and processing logic that facilitate navigation between maps and between individual spectra within each map. Hyperspectral map files are loaded and listed in the MapList widget. All individual spectra of the selected map are then listed in the SpectraList widget (right panel, Figure 2). The heatmap viewer displays by default the integrated intensity across the selected spectral range. The fitting workflow is identical to that of individual spectra (Section 3.1).

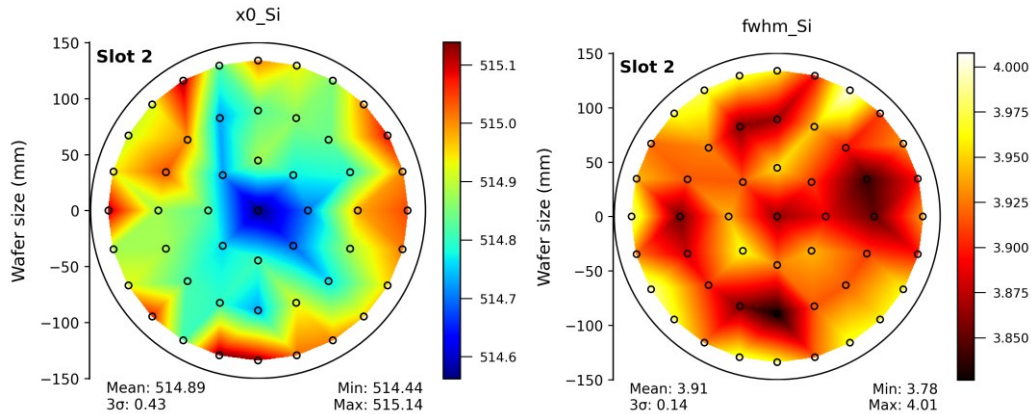


Figure 5: Wafer plot of (a) peak position and (b) peak FWHM with automatically computed statistical summaries (mean, min, max, and standard deviation). Multiple color palettes are available.

The VBF Engine processes the entire map as a single batched operation. After fitting, the Z-parameter dropdown is updated with the fitted parameters (e.g., peak position, FWHM, amplitude), and the heatmap is re-rendered to visualize spatial distributions of each fitted parameters of all peaks.

3.3. Plotting graphs

Once fit-result DataFrames have been collected and forwarded from the Spectra or Maps Workspace (Section 3.1 and 3.2), the user can switch to the Graphs Workspace for statistical visualization (Figure 6). DataFrames can also be imported directly from external Excel or CSV files.

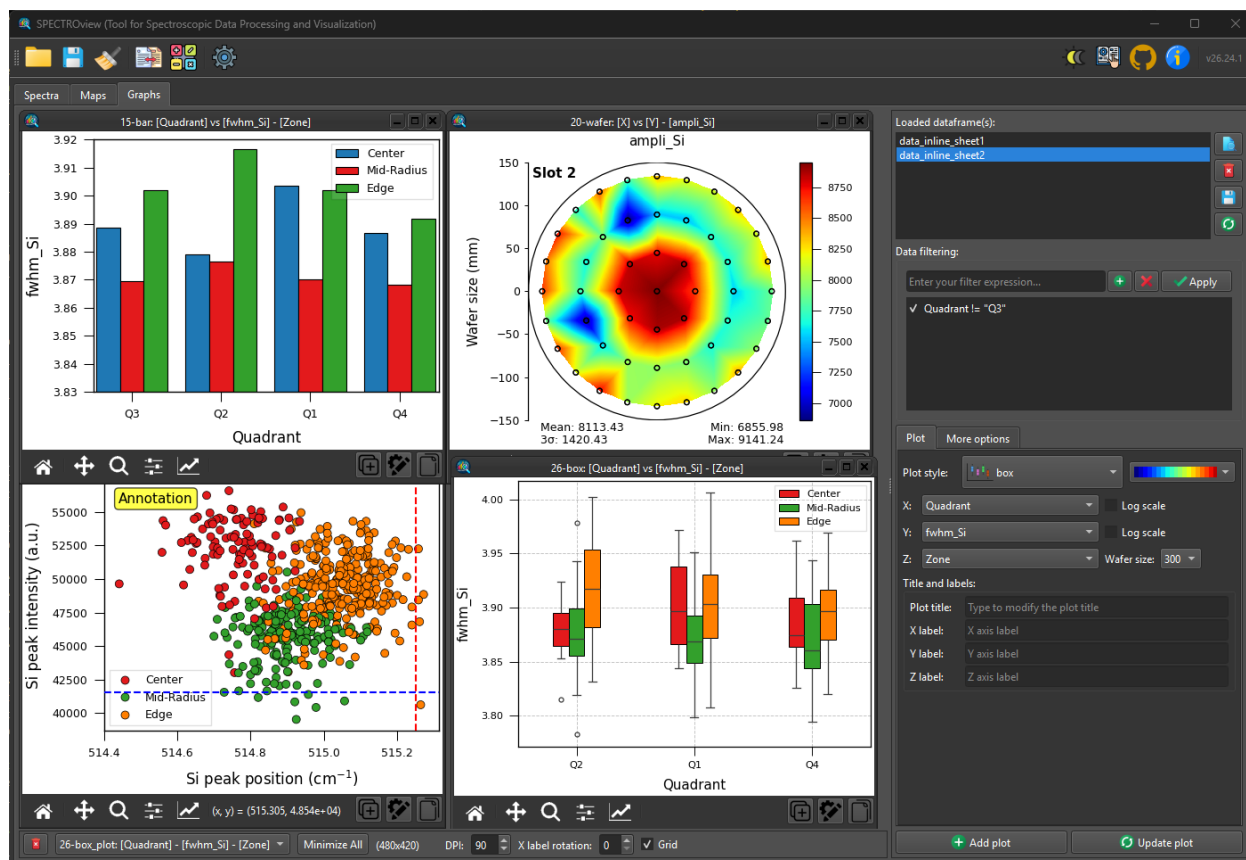


Figure 6: User interface of Graphs Workspace

The user selects a dataset from the list panel, assigns columns to the X, Y, and optionally Z axes via dropdown menus, and chooses a plot style (e.g., scatter, box, bar, line, trendline, wafer, or 2D map). Clicking "Add Plot" renders the figure in the MDI area. Multiple plots can be displayed simultaneously for side-by-side comparison.

Each plot can be refined through a customization dialog that provides control over axis limits, legend entries (label, color, marker), and annotations (vertical/horizontal reference lines, draggable text labels).

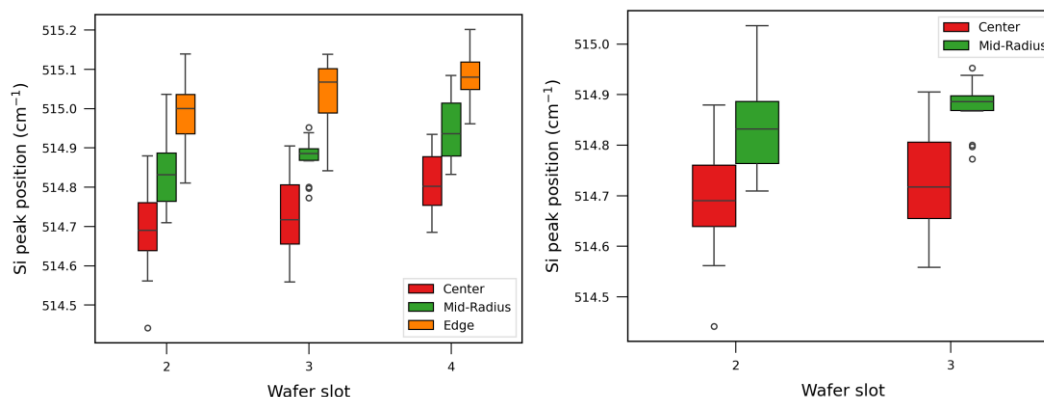


Figure 7: (a) Box plot plotted directly from best-fit results forwarded from Maps Workspace to show inter-wafers comparison of Si peak position, with data plotted divided three different region of wafer without any post-processing or data cleaning. (b) Same data with figure (a) but applying some filters to filter out wafer 4 and zone "Edge" from plot.

Figure 5 show an example of wafer plot of fitted parameters accompanied by automatically computed statistics (mean, max, min, standard deviation).

Figure 7a presents box plots allowing a comparison between data from three wafers. A dynamic filtering feature (cf. right-side panel in Figure 6) allows the user to subset the plotted data using boolean expressions applied to DataFrame columns, enabling rapid exploration of parameter dependencies without modifying the underlying data. For example, Figure 7b is obtained using same datasheet as Figure 7a, but applying two filters *Slot != "4"* and *Zone != "Edge"* to exclude wafer n° 4 and wafer Edge zone from the plot. More demos can be found [here](#).

4. Impact

4.1. Comparaison with some existing tools

Table 2 present comparison of SPECTROview with some existing spectroscopic analysis tools. SPECTROview distinguishes itself along two axes:

Table 2: Compares SPECTROview with representative open-source and proprietary tools for spectroscopic data analysis. NA = not applicable (library without fitting or proprietary implementation); ✓* = partial support (fityk uses analytical derivatives for some built-in functions).

Feature	SPECTROview	Fitspy	Fityk	RamanSPy	PyFasma	Wire/Labspec
Open source	✓	✓	✓	✓	✓	✗
Cross-platform	✓	✓	✓	✓	✓	✗
GUI for interactive fitting	✓	✓	✓	✗	✗	✓
Hyperspectral map support	✓	✓	✗	✓	✗	✓
Wafer map support	✓	✗	✗	✗	✗	✗
Batched vectorized fitting	✓	✗	✗	✗	✗	✓
Analytical Jacobians	✓	✗	✓*	NA	✗	NA

Integrated statistical plotting	✓	X	X	X	X	X
Inter-peaks parameter expression	✓	✓	✓	X	✓	X
MVA analysis	✓	X	X	✓	✓	✓

First, it is the only open-source tool that integrates the complete workflow from multi-format file import (Table 1) through interactive peak fitting to publication-quality statistical plotting within a single GUI application. Its ergonomic and highly optimized interface is designed to be accessible to a broad audience, including researchers, engineers, and students, who need an efficient platform for spectral data inspection, interactive processing and analysis of small or large-scale hyperspectral datasets without writing code, and especially with no licensing barriers.

Second, its VBF Engine introduces a high-performance computational approach that is, to our best knowledge, unique among open-source spectroscopy software. To appreciate this contribution, it is instructive to contrast the engine against existing computational paradigms:

- Traditional spectroscopy tools (Imfit, fitspy, fityk, PyFasma, RamanSPy): Existing open-source curve-fitting tools rely on sequential, per-spectrum optimization loops. Whether implemented via Python iteration (fitspy, PyFasma) or C++ optimization (fityk), they optimize one spectrum at a time. None of them formulate the fitting of N spectra as a single batched least-squares problem solved through vectorized array operations.
- Deep learning frameworks (PyTorch, TensorFlow): While batched Levenberg–Marquardt implementations exist in machine learning repositories, they rely on automatic differentiation (autograd) designed for neural network training. They are not tailored for evaluating closed-form spectroscopic line shapes (e.g., Gaussians, Lorentzians) or for managing independent per-spectrum convergence, where each spectrum may converge at a different iteration.

Table 3 : Comparative benchmarking timing results on different datasets tested on a PC with CPU core i5 8365U. depends on the size of hyperspectral data and the complexity of the fit model (peak number), SPECTROview can yields suppedups of upto ~ 60×.

	MoS ₂ flake	Array of pixel	4 x WaferMap	CL map
Map size (number of pixel)	1520	3721	196	16384
Number of peak(s)	3	1	6	1
Fitting time (s) : other*	22	31	33	192
Fitting time (s) : VBF engine	1.3	0.5	1.9	5.3
Performance boost	16×	62×	17×	36×

* Imfit sequential per-spectrum approach

The VBF Engine bridges this methodological gap by adapting the concept of batched optimization into a lightweight, pure-NumPy architecture. Rather than iterating over spectra in a Python loop, the engine assembles the N Jacobian matrices and normal-equation systems as three-dimensional arrays and solves them in a single vectorized LAPACK dispatch. Analytical Jacobians (derived in closed form for each supported line shape) replace the numerical finite-difference approximations used by most tools, further reducing computational cost. A per-spectrum convergence mask progressively excludes converged spectra from subsequent

iterations, ensuring that early convergers incur near-zero overhead. This fully vectorized strategy yields speedups of up to 60× compared to sequential per-spectrum methods (Table 3), even on a standard low-power CPU (Intel Core i5-8365U).

4.2 Reproducibility and availability

SPECTROview is released under the GNU General Public License v3.0 and hosted on GitHub at <https://github.com/CEA-MetroCarac/SPECTROview>. The repository includes source code, example datasets. The software is registered on PyPI and can be installed with a single command (*pip install spectroview*).

Comprehensive documentation is available at <https://CEA-MetroCarac.github.io/SPECTROview/> including a User Manual for standard users and a Developer Guide for users who want to contribute to project.

SPECTROview is used in laboratories at the Platform for Nanocharacterisation (PFNC) and the cleanrooms of CEA-LETI (Grenoble, France) for routine characterization of advanced semiconductor materials by Raman spectroscopy and/or photoluminescence^[15–18]. It is now also open to external users.

5. Conclusions

SPECTROview provides an integrated, open-source solution for spectroscopic data processing, curve fitting, and visualization. By combining interactive and highly optimized GUI-based analysis with a high-performance batched fitting engine, it addresses a practical gap in the available free open-source tooling for researchers working with spectroscopic and hyperspectral datasets. The MVVM software architecture, workspace persistence, and extensible peak-model registry facilitate both daily laboratory use and long-term reproducibility. Current development directions include support for additional spectroscopic file formats, GPU-accelerated tensor fitting, and enhanced multivariate analysis capabilities. Community contributions are welcome via the GitHub repository.

CRedit authorship contribution statement

- **Van Hoan Le:** Writing original draft, Writing review & editing, Testing, Validation, Conceptualization, Software.

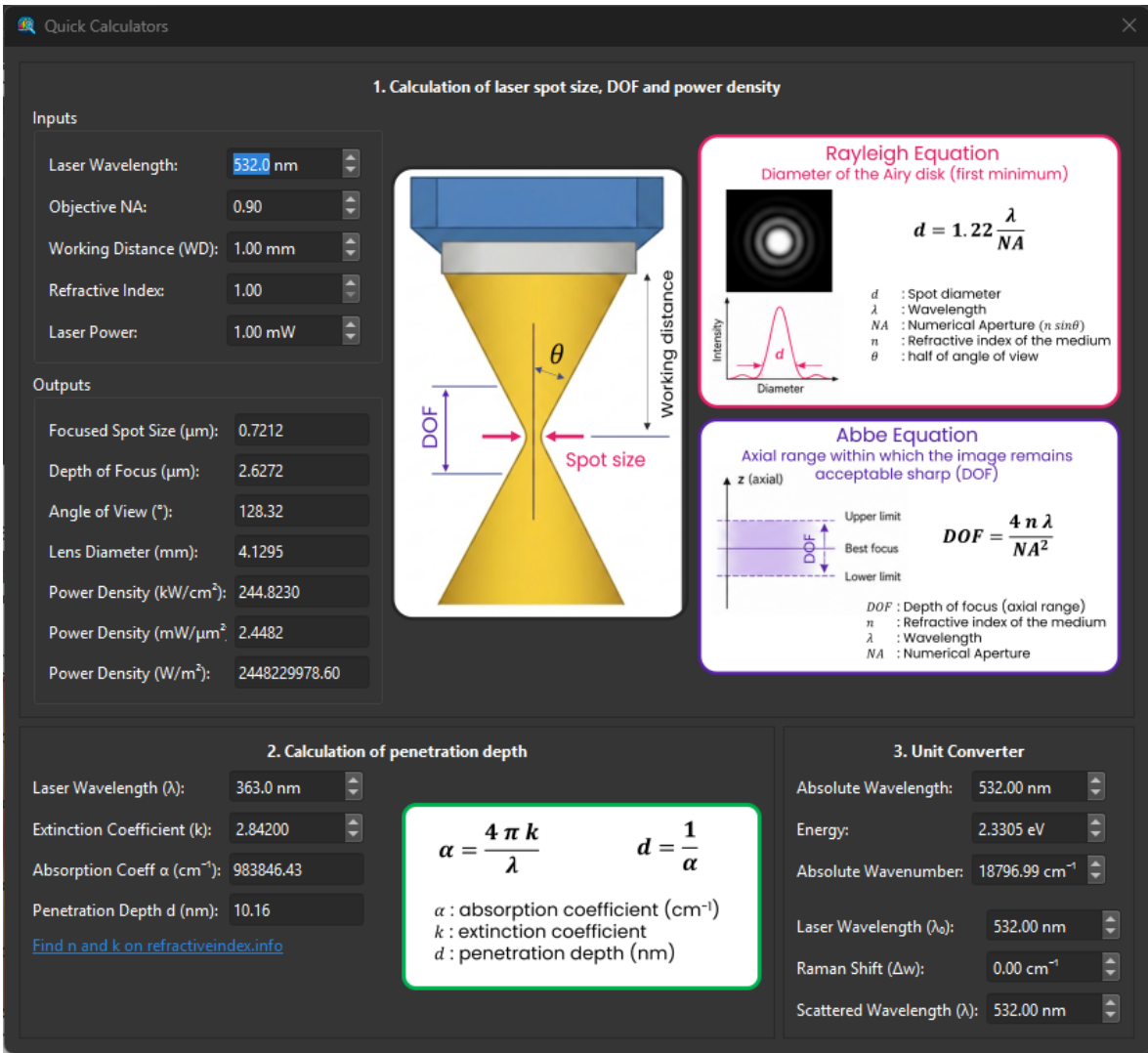
Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work, carried out on the Platform for Nanocharacterisation (PFNC), was supported by the “Recherche Technologique de Base” and “France 2030 - ANR-22-PEEL-0014” programs of the French National Research Agency (ANR). The authors gratefully acknowledge Patrick Quéméré for his assistance during the initial phase of the project.

Appendix A



Listing 1: User interface of Quick Calculators Utility. More detailed on this feature at Github documentation site: [here](#).

References

- [1] M. Wojdyr, *J. Appl. Crystallogr.* **2010**, *43*, 1126.
- [2] P. Quéméré, *J. Open Source Softw.* **2024**, *9*, 5868.
- [3] M. Newville, T. Stensitzki, D. B. Allen, A. Ingargiola, **2014**, DOI 10.5281/zenodo.11813.
- [4] D. Georgiev, S. V. Pedersen, R. Xie, Á. Fernández-Galiana, M. M. Stevens, M. Barahona, *Anal. Chem.* **2024**, *96*, 8492.
- [5] E. Pavlou, N. Kourkoumelis, *The Analyst* **2025**, *150*, 3112.
- [6] M. Toplak, S. T. Read, C. Sandt, F. Borondics, *Cells* **2021**, *10*, 2300.
- [7] The Qt Company, “Qt for Python (PySide6),” can be found under <https://doc.qt.io/qtforpython-6/>, **2024**.
- [8] D. Erb, **2026**.
- [9] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, *Nat. Methods* **2020**, *17*, 261.
- [10] D. W. Marquardt, *J. Soc. Ind. Appl. Math.* **1963**, *11*, 431.
- [11] E. Anderson, Z. Bai, C. Bischof, L. S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, D. Sorensen, *LAPACK Users’ Guide*, Society For Industrial And Applied Mathematics, **1999**.
- [12] M. L. Waskom, *J. Open Source Softw.* **2021**, *6*, 3021.
- [13] J. D. Hunter, *Comput. Sci. Eng.* **2007**, *9*, 90.
- [14] D. D. Lee, H. S. Seung, *Nature* **1999**, *401*, 788.
- [15] Z. Szekrényes, V.-H. Le, P. Bellanger, L. Badeeb, B. Éles, A. I. Faragó, E. Nolot, D. Barge, M. Gallard, J.-M. Hartmann, P. Hauchecorne, V. Loup, J. Sturm, A. Sediri, R. Bolla, R. Petrovski, B. Gombkötő, B. Somogyi, G. Nádudvari, A. Pongrácz, M. Dallery, N. Laurent, in *2025 36th Annu. SEMI Adv. Semicond. Manuf. Conf. ASMC*, **2025**, pp. 1–4.
- [16] N.-P. Tran, F. Milesi, V.-H. Le, L.-D. Mohgouk Zouknak, P. Dezest, Ph. Rodriguez, L. Brunet, B. Duriez, M.-C. Cyrille, C. Fenouillet-Beranger, *Solid-State Electron.* **2025**, *229*, 109196.
- [17] D. Barge, M. Gallard, J.-M. Hartmann, F. Fournel, V. Loup, F. Mazen, E. Nolot, P. Hauchecorne, J. Sturm, V. H. Le, I. Huyet, D. Delprat, F. Boedt, F. Servant, *Solid-State Electron.* **2026**, *232*, 109307.
- [18] D. Monteil, Y. Mazel, F. Deprat, E. Prevost, R. Duru, D. L. Cunff, E. Nolot, V. H. Le, P. Quéméré, M. Mermoux, **2026**.